Owi

Exécution symbolique parallèle cross-langage

Léo Andrès[1,3], Filipe Marques[2], Arthur Carcano[1],
Pierre Chambart[1], José Fragoso Santos[2], Jean-Christophe Filliâtre[3]

1. OCamlPro
2. INES-ID / Instituto superior Tecnico, University of Lisbon
3. Université Paris-Saclay, CNRS, ENS Paris-Saclay, Inria, Laboratoire Méthodes Formelles

2024-11-14

Wasm



Cible de compilation pour les navigateurs cherchant à combler les lacunes de JavaScript.

Version initiale:

- langage à **pile** ;
- **typé** (type de la pile connu statiquement à chaque instruction) ;
- **fonctions** ;
- une **mémoire linéaire** (un tableau d'octets);
- une sémantique formelle, sans UB.

Exemple de programme Wasm



```
(module
(func $f (param $n i32) (result i32)
                                                ;; []
 (i32.lt_s (local.get $n) (i32.const 2)) ;; [ n < 2 ]
 (if (then       ;; [ ]
   local.get $n ;; [ n ]
   return ))     ;; early return
                                               ;; [ ]
 (i32.sub (local.get $n) (i32.const 2)) ;; [ n-2 ]
 call $f                                ;; [ f(n-2) ]
 (i32.sub (local.get $n) (i32.const 1)) ;; [ n-1; f(n-2) ]
 call $f                                ;; [ f(n-1); f(n-2) ]
 i32.add                                ;; [ f(n-1) + f(n-2)]
))                                      ;; implicit return
```

Compilation vers Wasm



Version initiale: fonctionne bien pour les langages sans GC.

Compiler un langage avec GC:

- réécrire l'**environnement d'exécution** en Wasm ;
- impossible pour le GC de **scanner** la pile ;
- nécessite une **shadow stack** (lent, choix fait par Haskell) ;
- problème d'interaction avec le GC de l'hôte (JS dans les navigateurs).

Consensus quant à la nécessité d'ajouter un GC à Wasm.

Sujet de thèse initial : compilation d'OCaml vers WasmGC.

Compilation d'OCaml vers WasmGC



Plusieurs extensions nécessaires:

- Typed Function References + WasmGC ;
- Tail-Call ;
- Exceptions (optionnel).

À l'époque, aucun environnement d'exécution Wasm n'implémente le GC et les appels terminaux à la fois.

Développement d'**Owi**, un interpréteur Wasm en OCaml, pour expérimenter avec ces extensions.

En parallèle, développement de **Wasocaml**, compilateur OCaml vers WasmGC.

Dagstuhl Mars 2023



En mars 2023, présentation de Wasocaml à Dagstuhl.

> Wasocaml is nice but what about making a **symbolic interpreter** with Owi?
>
> — José Fragoso Santos

Exécution symbolique



De quoi s'agit-il ?

- une analyse statique ;
- explore **tous les chemins** d'exécution ;
- collecte des informations sur les branchements pris ;
- utilise un solveur SMT pour tester l'atteignabilité des branches et générer des modèles.

Le but: trouver les valeurs d'entrées du programme menant à un **trap** ou à une violation d'assertion.

Méthode **automatique** (≠ vérification déductive) et **complète** (≠ interprétation abstraite) mais pas **correcte**: pas de faux positifs, possiblement des faux négatifs (si l'analyse ne termine pas).

Symboles et harnais



```
(module
  (import "sym" "i32_symbol" (func $i32_symbol (result i32)))

  (func $f (param $x i32)
    (if (i32.lt_s (i32.const 5) (local.get $x))
    (then unreachable))
  )

  (func $start
    call $i32_symbol ;; [ x ]
    call $f
  )

  (start $start))
```

Modèles



Si un **trap** est atteint, on demande un modèle au solveur SMT. Cela nous donne les valeurs d'entrées qui vont faire échouer le programme:

```
$ owi sym ./mini.wat
Trap: unreachable
Model:
  (model
    (symbol_0 (i32 6)))
Reached problem!
[13]
```

Implémentation concrète



Implémentation initiale:

```
match instr, stack with
  Binop Add, x::y::stack -> (x + y) :: stack
| Binop GT,  x::y::stack -> (x > y) :: stack
| If_then_else (if_top, if_bot), cond::stack ->
  if cond then eval if_top stack
          else eval if_bot stack
```

Comment implémenter une version symbolique en gardant la verison concrète ?

Ou: comment rendre un interpréteur symbolique facilement ?

Étape 1/2: abstraire le type des valeurs



```
module type Values = sig
  type t
  val add : t -> t -> t
  val gt : t -> t -> t
end
```

```
| Binop Add, x::y::stack -> (Value.add x y) :: t
| Binop GT,  x::y::stack -> (Value.gt x y) :: t
```

Dans le cas concret `t` reste une valeur wasm concrète: `42`.

Dans le cas symbolique: `t` est une **expression** avec des symboles:

`x < 42 && y = x || y = 22`.

Étape 2/2: abstraire sur la stratégie d'exécution



```
module type Choice = sig
  type 'a t
  val return : 'a -> 'a t
  val bind : 'a t -> ('a -> 'b t) -> 'b t
  val select : Value.bool -> bool t
end
```

```
| If_then_else (if_top, if_bot), cond::stack ->
  let* cond = Choice.select cond in
  if cond then eval if_top stack
          else eval if_bot stack
```

La majorité du code est inchangée. Il faut seulement insérer des `select` + `bind` aux points de branchement.

Implémentation de Choice



Cas concret: identité (pas de coût à l'exécution avec Flambda et les bonnes annotations d'inlining).

Cas symbolique: évalue les deux branches, stocke l'état du programme et de la condition de chemin.

Version symbolique va évaluer toutes les branches **en parallèle** grâce à OCaml 5.

Implémentation basée sur trois couches de monades:

- monade d'erreur ;
- monade d'état ;
- monade de coroutines coopératives scissionables.

Exécution symbolique de code C



```c
#include <owi.h>
void check_function_equivalence(f1, f2) {
  unsigned int x = owi_i32();
  unsigned int y = owi_i32();
  owi_assert(f1(x, y) == f2(x, y));
}
unsigned int mean1(unsigned int x, unsigned int y) {
  return (x & y) + ((x^y) >> 1); }
unsigned int mean2(unsigned int x, unsigned int y) {
  return (x + y) / 2; }
void main(void) {
  check_function_equivalence(mean1, mean2);
}
```

Exécution symbolique de code C



```
$ owi c ./function_equiv.c --fail-on-assertion-only -O0
Assert failure: (bool.eq (i32.add (i32.and symbol_0 symbol_1)
(i32.shr_u (i32.xor symbol_0 symbol_1) (i32 1))) (i32.shr_u
(i32.add symbol_0 symbol_1) (i32 1)))
Model:
  (model
    (symbol_0 (i32 -922221680))
    (symbol_1 (i32 1834730321)))
Reached problem!
```

Exécution symbolique cross-language



Version C:

```c
float dot_product(float x[2], float y[2]) {
    return (x[0]*y[0] + x[1]*y[1]);
}
```

Version Rust:

```rust
fn dot_product_rust(x: &[f32; 2], y: &[f32; 2]) -> f32 {
    x.iter().zip(y).map(|(xi, yi)| xi * yi).sum()
}
```

Exécution symbolique cross-language

Glue 1/2:



```
extern "C" {
    pub fn dot_product(x: *const f32, y: *const f32) -> f32;
}

fn dot_product_c(x: &[f32; 2], y: &[f32; 2]) -> f32 {
    unsafe { dot_product(x.as_ptr(), y.as_ptr()) }
}
```

Exécution symbolique cross-language

Glue 2/2:



```rust
fn main() {
    let x = std::array::from_fn(|_| owi_sym::f32_symbol());
    let y = std::array::from_fn(|_| owi_sym::f32_symbol());
    let c_val = dot_product_c(&x, &y);
    let rust_val = dot_product_rust(&x, &y);
    owi_sym::assert(
      (c_val.is_nan() && rust_val.is_nan()) ||
      (c_val.to_bits() == rust_val.to_bits())
    )
}
```

Exécution symbolique cross-language



Étonnamment, Owi trouve un contre-exemple :

```
Model:
  (model
    (symbol_0 (f32 -0.))
    (symbol_1 (f32 -0.))
    (symbol_2 (f32 0.))
    (symbol_3 (f32 0.)))
```

Exécution symbolique cross-language



Rust utilise comme accumulateur initial `0.`. Or, d'après la norme des flottants, `-0. + +0. = +0.` (le véritable élément neutre est `-0.` et pas `+0.`). En Rust, on obtient donc `+0.` tandis qu'en C on a `-0.` puisqu'il n'y a pas d'accumulateur dans la version sans itérateur.

PR pour corriger la stdlib de Rust acceptée. L'accumulateur initial est maintenant `-0.`.

Cela a cassé **typst**, l'outil utilisé pour faire ces slides et qui dépendait de ce comportement.

Évaluation expérimentale Wasm



n_o	$n_u = 1$					$n_u = 2$					$n_u = 3$				
	$s_{\texttt{Owi}-24}$	$S_{\texttt{Mcore}}$	$S_{\texttt{SW}}$	$S_{\texttt{WASP}}$	$S_{\texttt{Owi}-1}$	$s_{\texttt{Owi}-24}$	$S_{\texttt{Mcore}}$	$S_{\texttt{SW}}$	$S_{\texttt{WASP}}$	$S_{\texttt{Owi}-1}$	$s_{\texttt{Owi}-24}$	$S_{\texttt{Mcore}}$	$S_{\texttt{SW}}$	$S_{\texttt{WASP}}$	$S_{\texttt{Owi}-1}$
2	1.0	17.2	2.5	0.4	0.6	1.0	122.2	15.8	1.4	1.5	1.0	635.7	64.5	5.0	5.7
3	1.0	33.1	5.3	0.5	0.6	1.0	281.3	33.2	2.4	2.9	1.0	842.7	89.7	7.8	8.9
4	1.0	48.3	10.0	0.9	0.9	1.0	444.1	52.6	3.7	4.9	1.0	844.3	101.6	9.1	10.8
5	1.0	75.7	13.4	1.4	1.6	1.0	589.2	69.0	5.0	7.2	1.0	772.3	92.7	10.5	12.2
6	1.0	134.6	20.3	1.7	2.2	1.0	647.0	88.1	5.9	9.1	1.0	688.8	98.8	12.3	13.1
7	1.0	170.5	35.3	2.5	3.4	1.0	626.7	96.9	6.1	9.8	1.0	629.6	94.0	16.6	14.0
8	1.0	201.0	40.5	2.9	4.1	1.0	597.1	95.3	6.9	10.7	–	–	–	–	–
9	1.0	212.2	47.2	3.4	5.1	1.0	564.0	89.1	6.8	11.2	–	–	–	–	–

- Manticore: Owi `17,2` à `844,3` plus rapide avec une moyenne de `312,6`.
- SeeWasm : Owi `2,5` à `101,6` plus rapide avec une moyenne de `57,1`.
- WASP : Owi `0,4` à `16,4` plus rapide avec une moyenne de `4,1`.
- Owi1 : Owi `0,6` à `14,0` plus rapide avec une moyenne de `4,5`.

Évaluation expérimentale C



Tool	Reached	Timeout	Nothing	$\frac{\text{Nothing}}{\text{Reached+Nothing}}$
KLEE	782	368	65	7.67%
Owi	676	539	0	0.00%
Symbiotic	489	657	69	12.37%

Owi ne fait aucune approximation et ne manque aucun bug si l'analyse termine (modulo les choix faits par le compilateur C quant aux UB).

Contrairement aux autres, Owi n'implémente pas encore un modèle mémoire sophistiqué ou une heuristique d'exploration des chemins.

Des annotations à l'exécution symbolique



```
/*@ requires 2 <= n <= MAX_SIZE;
    requires \valid(is_prime + (0 .. (n - 1)));
    ensures
      \forall integer i; 0 <= i < n ==> ... */
void primes(bool *is_prime, int n) {
  for (int i = 0; i < n; ++i) { is_prime[i] = true; }
  for (int i = 2; i * i < n; ++i) {
    if (!is_prime[i]) continue;
    for (int j = i; i * j < n; ++j) { is_prime[i * j] =
false; } } }
```

L'option `owi c --e-acsl` permet d'exécuter symboliquement du code annoté par des spécifications en générant du code avec des assertions via E-ACSL et en utilisant notre propre variante du runtime E-ACSL.

Des annotations à l'exécution symbolique



Nous avons fait la même chose en Wasm (stage de Zhicheng Hui):

```
(module

  (@contract $sum
    (ensures (= result (+ $p1 (+ $p2 (+ $p3 $p4))))))
  (func $sum (param $p1 i32) (param $p2 i32) ...
    ;; ...
  ))
```

Conception de **Weasel**: WEbAssembly SpEcification Language. Le premier langage de spécification conforme au standard. On génère des assertions à partir de celui-ci, comme le fait E-ACSL. Permet par exemple de vérifier des runtimes écrits en Wasm.

Owi comme moteur de programmation par contrainte



```c
#include <owi.h>

int main() {
  int x = owi_i32();
  int x2 = x * x;
  int x3 = x * x * x;

  int a, b, c, d = 1, -7, 14, -8;
  int poly = a * x3 + b * x2 + c * x + d;

  owi_assert(poly != 0);

  return 0;
}
```
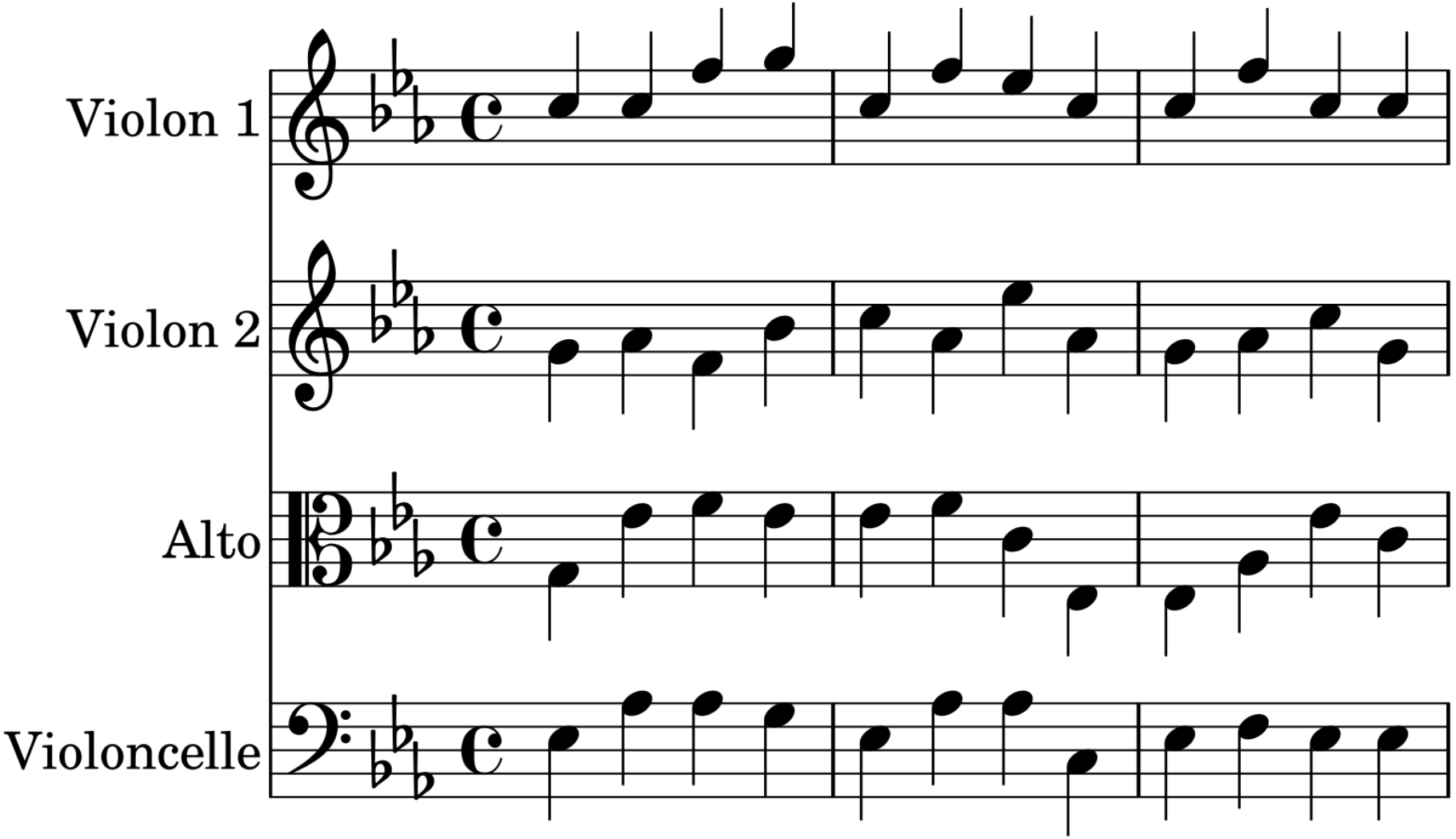
Owi comme moteur de programmation par contrainte



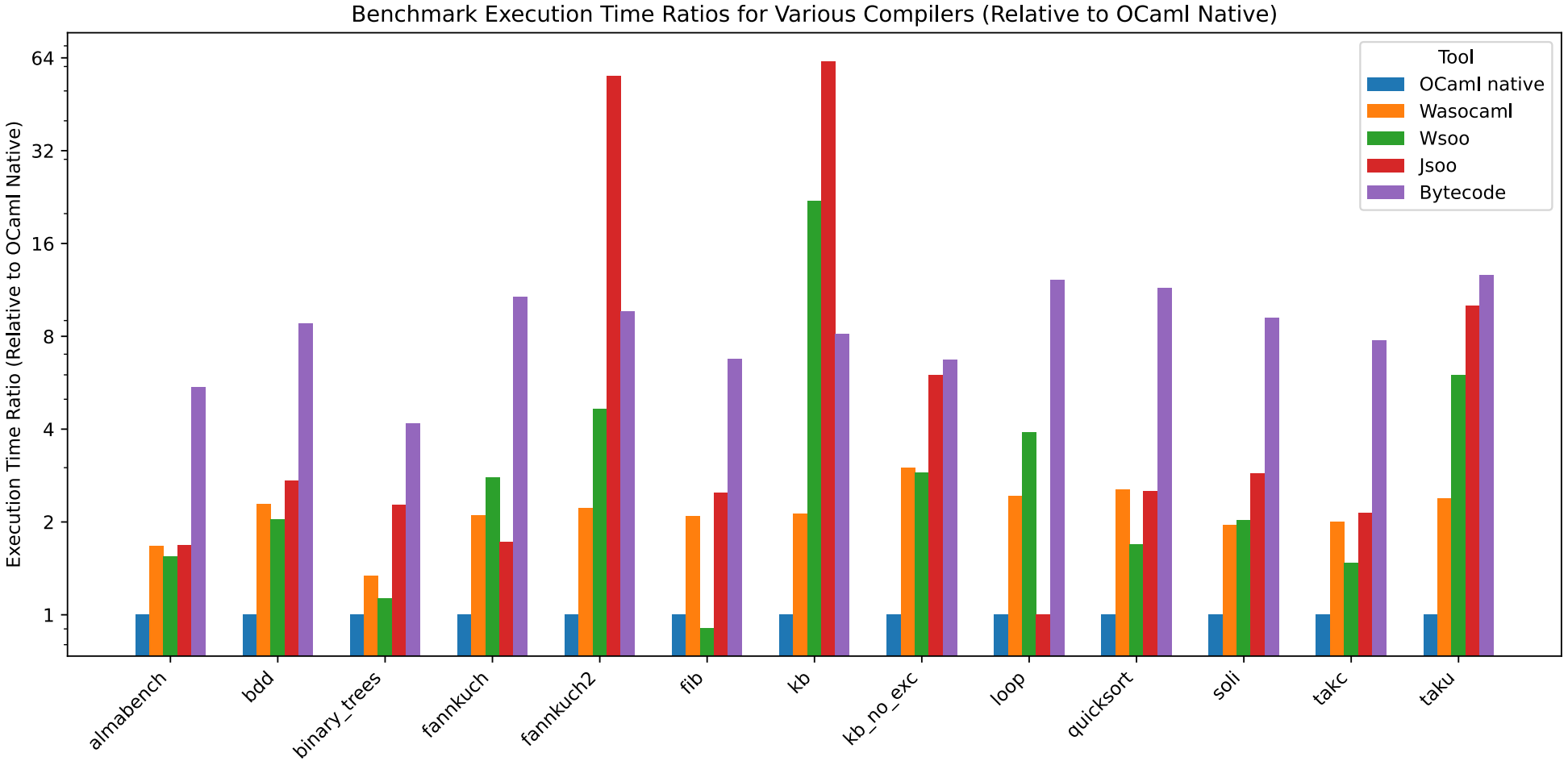
Utilisé pour:

- résoudre un labyrinthe ;
- générer des cartes pour le jeu Dobble ;
- générer des partitions pour quatuor à cordes respectant des règles de base d'harmonie…

Owi comme moteur de programmation par contrainte



Vers l'exécution symbolique d'OCaml ?



Vers l'exécution symbolique d'OCaml ?



Outil	Minimum	Maximum	Moyenne	Médiane	Écart-type
Wasocaml	1.34	3.00	2.17	2.13	0.39
Wasm_of_ocaml	0.91	21.99	4.08	2.04	5.36
Js_of_ocaml	1.00	62.48	11.84	2.51	20.36
Bytecode	4.17	12.61	8.75	8.82	2.48

Wasocaml est un prototype fonctionnel et présente de bonnes performances **prédictibles**.

Implémentation de WasmGC encore incomplète dans Owi…

Travaux futurs



Beaucoup d'idées, principalement:

- implémentation d'heuristiques de parcours dans Owi;
- implémentation de WasmGC dans Owi;
- porter Wasocaml de Flambda1 à Flambda2 et compléter le runtime;
- exécution symbolique de code OCaml + C.

Un papier publié avec plus de détails: **Owi: Performant Parallel Symbolic Execution Made Easy, an Application to WebAssembly**.

Manuscrit de thèse disponible en ligne avec encore plus de détails (en français).

Merci !